

Terraform

UNIT 3

Lucas Bonanni

2024-06-10

Terraform

Contenido:

- Introduction to Terraform
- Core
- Providers
- Resources
- Variables & Outputs
- Modules
- State management
- Workspaces
- Security

Introduction to Terraform

Introduction to Terraform

Terraform, desarrollado por HashiCorp, es una herramienta de **Infraestructura como Código (IaC)** que permite a los ingenieros definir, aprovisionar y gestionar infraestructura de TI complejas de forma declarativa, usando un lenguaje de configuración sencillo. Es multiplataforma y admite múltiples proveedores de servicios en la nube.

How does Terraform work?

Crea y maneja recursos en las plataformas cloud y otros servicios a través del uso de su API.

Providers habilita a Terraform a trabajar virtualmente con “cualquier” plataforma o servicio de cloud que tenga disponible una API.



How does Terraform work?

Write

Definir recursos, pueden ser recursos multi clouds, de distintos providers y servicio. Por ejemplo, podemos crear una configuración para hacer un deploy de una aplicación en servidor, que por encima tiene configurada una VPC con security groups y un load balancer.

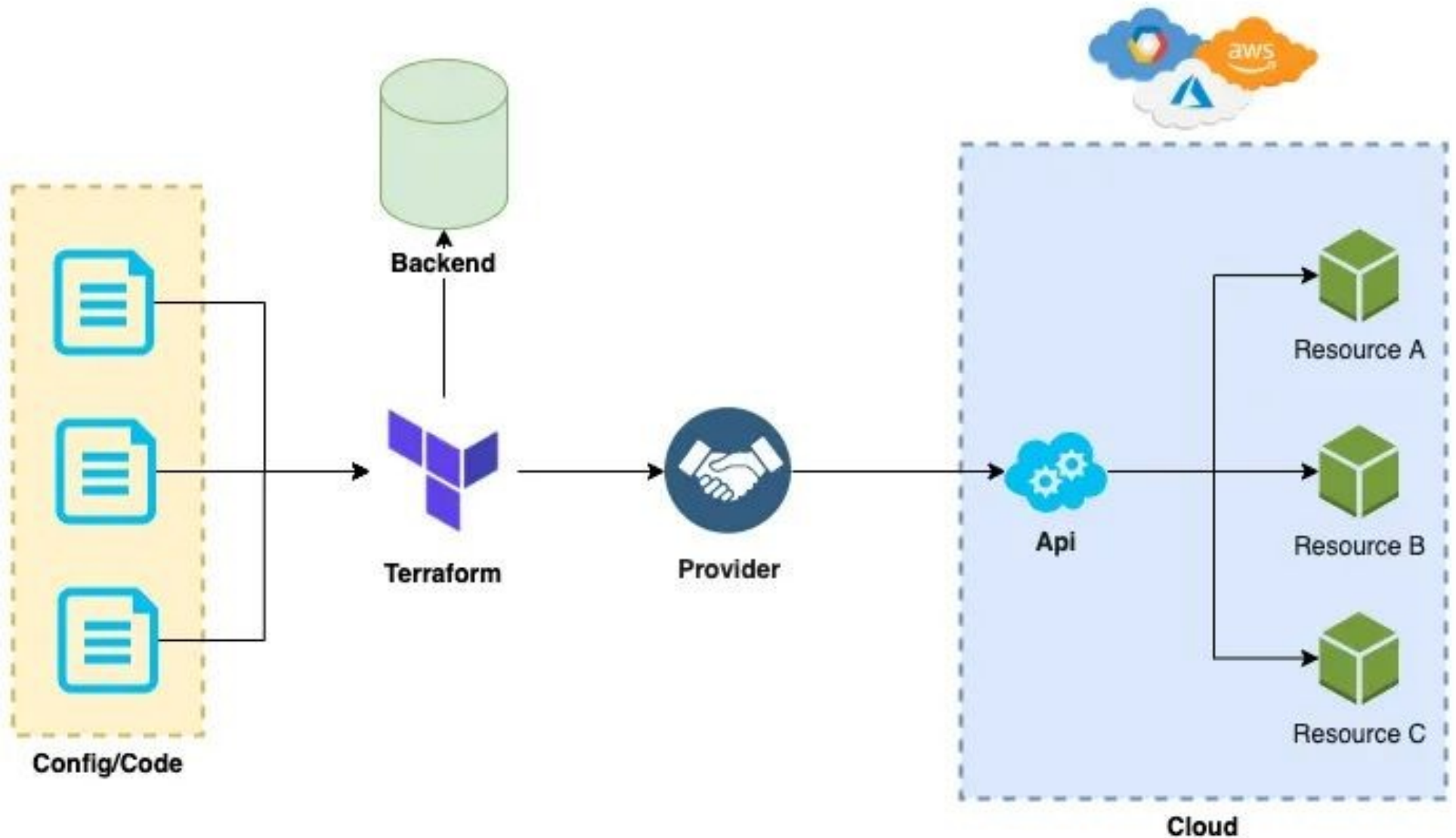
Plan

Terraform crea un “plan execution” describiendo la infraestructura que va a crear, upgradear o destruir basado en la infraestructura existente y tu configuración.

Apply

Terraform ejecuta la configuración indicada, tomando en cuenta el orden de las ejecuciones ya que pueden tener dependencias entre sí.

How does Terraform work?



Terraform Architecture

Componentes principales:

- **Core:** Motor de Terraform
- **Providers:** Plugins para interactuar con APIs
- **State:** Archivo que mantiene el estado de la infraestructura
- **Backend:** Donde se almacenan los archivos de estado

Track your Infrastructure

Ante cada cambio, Terraform genera un plan y puedes visualizar los cambios que se generarán con tu aprobación.

Usando un **state file**, Terraform mantiene tracking de tu infraestructura real.

Basado en ese state file, es que va a generar un plan cuando requieras una actualización o creación de nueva infra.

Standardize Configurations

Terraform soporta reutilización de componentes de configuraciones llamado “**module**”

Podemos utilizar configuraciones estándares que se encuentran en “**Terraform Registry**”
o escribir módulos estándares a demanda.

Core

Provider

Provider es un tipo de plugin en Terraform el cual se encarga de interactuar con cloud providers, SaaS providers o otras APIs.

Cada provider brinda un set de tipo de recursos y de fuentes de información que Terraform puede controlar.

Los Provider están separados de la herramienta de Terraform en sí y tienen su propio versionado, estos Providers pueden ser creados por terraform, empresas o propios.

Resources

Los recursos de Terraform es un elemento fundamental para describir uno o más objetos que serán usados por la infraestructura.

Puede ser:

- Virtual networks
- Servidores
- Base de datos
- DNS
- Etc.

Ejemplo:

```
1 resource "aws_instance" "web" {  
2   ami          = "ami-0c55b159cbfafa1f0"  
3   instance_type = "t2.micro"  
4 }
```



```
const pluckDeep = key => obj => key.split('.').reduce((accum, key) => accum[key], obj)  
  
const compose = (...fns) => res => fns.reduce((accum, next) => next(accum), res)  
  
const unfold = (f, seed) => {  
  const go = (f, seed, acc) => {  
    const res = f(seed)  
    return res ? go(f, res[1], acc.concat([res[0]])) : acc  
  }  
  return go(f, seed, [])  
}
```

Variables and Outputs

Variables

Permiten parametrizar las configuraciones de Terraform para hacerlas más flexibles y reutilizables.

```
1 variable "instance_type" {  
2   description = "Type of EC2 instance"  
3   type        = string  
4   default     = "t2.micro"  
5 }
```

Outputs

Permiten exportar información sobre los recursos creados.

```
1 output "instance_ip" {  
2   value = aws_instance.web.public_ip  
3 }
```

Modules

Los módulos en Terraform permiten agrupar y encapsular recursos y configuraciones para ser reutilizados.

Los módulos pueden ser versionados y compartidos, lo que facilita la colaboración y la reutilización de código entre proyectos y equipos.

Terraform Registry ofrece módulos comunitarios y oficiales para uso público.

State Management

El estado en Terraform es un archivo JSON que contiene información sobre los recursos creados y sus configuraciones.

Terraform utiliza el estado para:

- Mapear los recursos reales a las configuraciones
- Identificar los cambios necesarios para alcanzar el estado deseado

El estado puede ser almacenado localmente o en **backends remotos** para colaboración

Workspaces and Environment

Los **workspaces** permiten a los usuarios gestionar múltiples instancias de una configuración de Terraform.

Esto es útil para separar entornos de desarrollo, prueba y producción, cada uno con su propio conjunto de configuraciones y estado.

Los workspaces ayudan a evitar conflictos y a gestionar recursos de manera aislada.

Workspaces - Use Cases

Principales casos de uso:

- **Entornos separados:** Desarrollo, Testing, Producción
- **Versiones de infraestructura:** Mantener versiones en paralelo
- **Multi-clientes o multi-proyectos:** Separación entre clientes
- **Pruebas de carga:** Testing sin afectar producción

Security and Compliance

Terraform permite a los usuarios definir políticas de seguridad y compliance como código integrándose con herramientas como **Sentinel de HashiCorp** para la evaluación y el enforcement de políticas.

Esto facilita la automatización segura y el cumplimiento de normativas en entornos complejos y regulados.

Terraform in the Real World

Terraform se utiliza en una variedad de escenarios, desde la creación de infraestructura básica hasta la automatización de workflows complejos en entornos multi-cloud y híbridos.

Su ecosistema en crecimiento y la comunidad activa proporcionan recursos, herramientas y soporte para implementaciones exitosas.

Terraform - Flujo de Trabajo Típico

1. **Write:** Escribir la configuración de infraestructura
2. **Init:** Inicializar el directorio de trabajo (`terraform init`)
3. **Plan:** Crear un plan de ejecución (`terraform plan`)
4. **Apply:** Aplicar los cambios (`terraform apply`)
5. **Destroy:** Eliminar recursos cuando sea necesario (`terraform destroy`)

Beneficios de Terraform

- **Infraestructura como Código:** Versionable, reutilizable, documentada
- **Multi-Cloud:** Soporte para múltiples proveedores
- **Declarativo:** Define el estado deseado, no los pasos
- **Plan de Ejecución:** Previsualización de cambios antes de aplicar
- **Gráfico de Recursos:** Gestión automática de dependencias
- **Modularidad:** Código reutilizable mediante módulos

Clase Siguiente

Monitoring

Herramientas y prácticas para monitorear infraestructura y aplicaciones